Intel Xeon Phi: Common Optimizations

Sparsh Mittal

© Sparsh Mittal

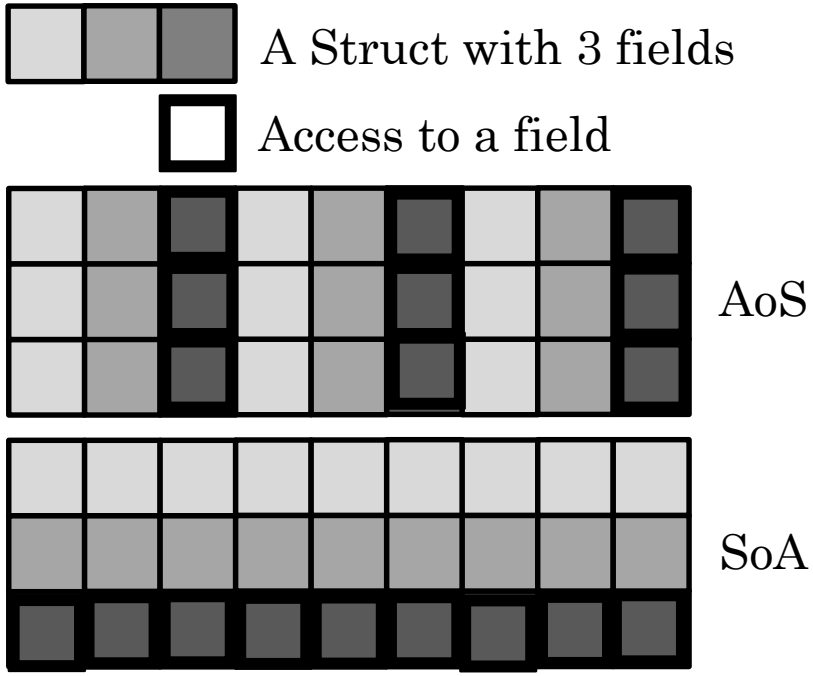
AoS to SoA (1 of 3)

(a) Code using AoS

```
struct Record{
  string name, age, ID;
};
Record Data[N];
for(i=0; i<N; i++)
  Process(Data[i].ID);
```

(b) Code using SoA

```
struct DataType{
string NameData[N];
string AgeData[N];
string IDData[N];};
DataType myData;
for(i=0; i<N; i++)
Process(myData.IDData[i]);
```

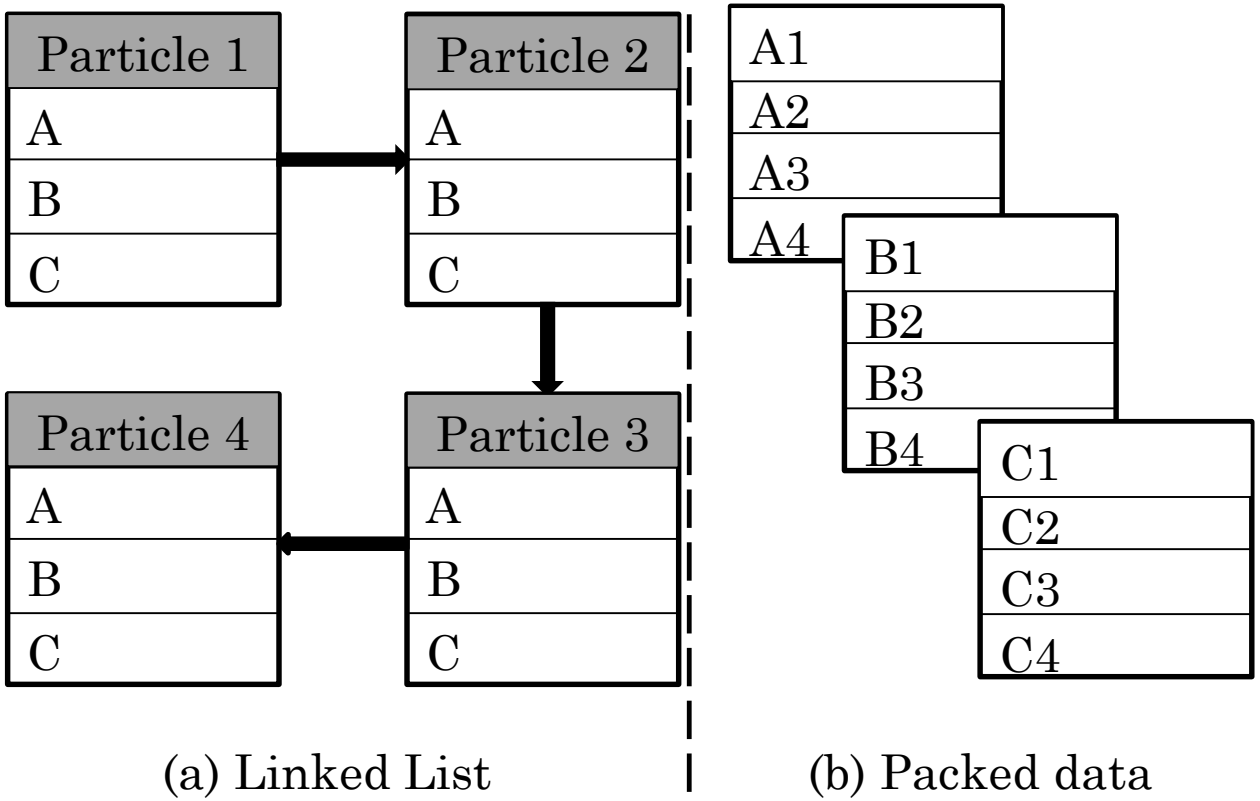


(c) Memory layout (assuming row-major order) and access pattern

AoS = array of structure, SoA = structure to array

© Sparsh Mittal

AoS to SoA (2 of 3)



© Sparsh Mittal

AoS to SoA (3 of 3)

- **AoS layout** shows poor performance on accessing same field of different records, since it gathers non-contiguous data which may lie in different cache-lines.
- This degrades cache and bandwidth efficiency and reduces the efficacy of vectorization.

- **With SoA layout**, the memory accesses can be satisfied from the contiguous regions of memory
- This facilitates vectorization and improves performance

© Sparsh Mittal

Loop collapsing

Original Loops

```
for i = 0 to N
  for j = 0 to M
    A[i][j] = C* A[i][j]
```

After Loop collapsing

```
for k = 0 to N*M
    NewA[k] = C* NewA[k]
```

- Loop collapsing combines multiple levels of a loop nest which increases the number of iterations in the for loop.
- This reduces loop overhead and allows parallelization over higher number of threads.

© Sparsh Mittal

Loop splitting (1 of 2)

- For nested loops, compiler vectorizes only the innermost level of the loop
- If compiler were to vectorize outer level of a nested loop, the vectorized code would include conditional branching instructions of the inner loop also => poor performance due to ineffective vectorization
- Due to this, compiler does not vectorize any instructions contained in the nested loop outside its innermost loop.

© Sparsh Mittal

Why inner loop is vectorized by default

Original code

```
for(i=0; i<N; i++)
   for(j=0; j<M; j++)
    Out[i][j]=In[i][j]+i+j;
```

Writing equivalent code for inner for loop

```
for(i=0; i<N; i++){
    j=0;
    ConditionCheck:
    if(j<M) {
     Out[i][j]=In[i][j]+i+j;
     j++;
     goto ConditionCheck;
    }
}
```

On vectorizing outer for loop, the code has control flow. As said earlier, code with control flow is difficult to vectorize.

On vectorising inner for loop, one gets unit-stride load/store for arrays Out and In. This is not true on vectorizing outer for loop

https://software.intel.com/en-us/articles/outer-loop-vectorization

© Sparsh Mittal

Loop splitting (2 of 2)

```
for i = 0 to NW_Size do
    #pragma ivdep
    for j = 0 to Synapse_Count do
        incoming_current[i][j] = calculate_synapse(i, j);
    end for
    calculate_new_state(incoming_current[i], i);
end for
```

```
for i = 0 to NW_Size do
    #pragma ivdep
    for j = 0 to Synapse_Count do
        incoming_current[i][j] = calculate_differential(i, j);
    end for
end for
#pragma ivdep
for i = 0 to NW_Size do
    calculate_new_state(incoming_current[i], i);
end for
```

- To vectorized entire code, split the loop in two loops. First is a two-layer loop and second is a single-layer loop
- Two loops are then vectorized separately.
- This also allows for exclusive modifications in each loop's code

© Sparsh Mittal

Programmer-assisted dependency disambiguation

- To be conservative, a compiler does not vectorize a code if it assumes presence of data-dependencies.
- E.g., if two pointers are dereferenced in the same loop, compiler may not be able to ascertain whether the memory regions pointed to by them are entirely distinct.
- This is especially true for dynamically allocated data.

Original Code

```
int *a, *b;
...

for i = 0 to N
   b[i] = a[i] * C;
```

After Dependency Disambiguation

```
int * restrict a, * restrict b;
...
#pragma ivdep
for i = 0 to N
   b[i] = a[i] * C;
```

See "Optimizing Extended Hodgkin-Huxley Neuron Model Simulations for a Xeon/Xeon Phi Node" paper for more details

© SparseVitap

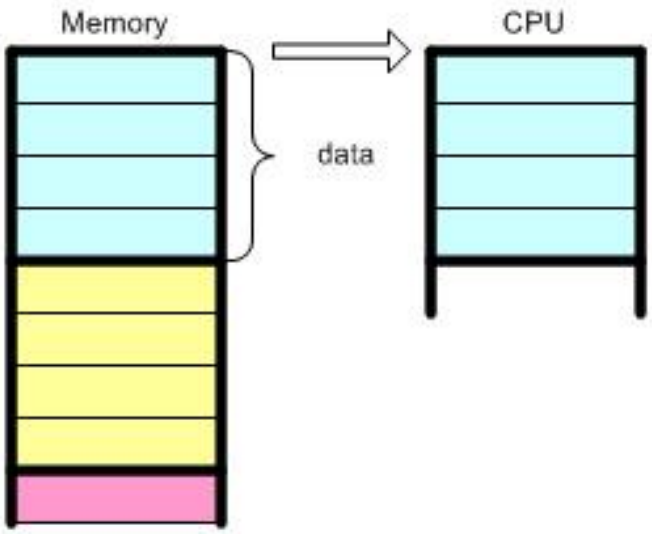
Programmer-assisted dependency disambiguation

- Sometimes, user knows the code and is certain that assumed dependencies can be ignored,
  - In example above, this means, memory spaces of pointers a and b are separate.
- If so, user can instruct the compiler to proceed with vectorization
- For this, use **#pragma ivdep** before for loop
- **restrict** keyword: promise to compiler that over the scope of the pointer, target of the pointer will only be accessed through that pointer (and pointers copied from it). guarantees exclusive memory accessing.

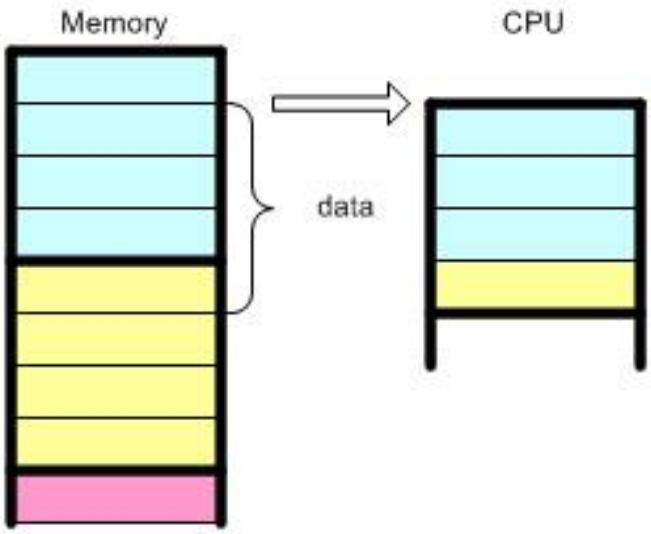
© Sparsh Mittal

Data alignment (1 of 2)

Data-alignment ensures that memory allocated to a structure begins at an address that is also the beginning of a cache line.
This avoids fetching of multiple cache lines for execution of a single SIMD instruction.



4-byte memory access for aligned data



4-byte memory access for misaligned data

© Sparsh Mittal

Data alignment (2 of 2)

- To ensure aligned memory allocations, avoid using standard C **malloc** function
- Instead, use Intel's **_mm_malloc**.
- This icc-compatible function ensures that data allocation will begin at an address divisible by the size of system's cache line (64B on Phi).

```
int *a = _mm_malloc(upper_bound*sizeof(int), 64);
int *b = _mm_malloc(upper_bound*sizeof(int), 64);
...
#pragma ivdep
for i = 0 to upper_bound do
    b[i] = a[i] * constant_k;
end for
```

© Sparsh Mittal

Prefetching

- On Phi, SW prefetching prefetches all regular access patterns that can be analyzed by the compiler
- L2 cache has a simple hardware prefetcher

© Sparsh Mittal